

Project Report

DSA Visual Lab

An interactive educational website for visually understanding Data Structures and Algorithms.

Student Name	Aikansh
Registration Number	12303820
Class Section	9P196
Project ID	WPEP-26-284
Live Website	https://aikansh-official.github.io/DSA-Visual-Lab/
GitHub Repository	Aikansh-Official/DSA-Visual-Lab

Prepared as a web project submission demonstrating visual learning, frontend engineering, and practical deployment using GitHub Pages.

1. Abstract

DSA Visual Lab is a web-based learning platform designed to help school students understand Data Structures and Algorithms through direct visual interaction. Instead of beginning with code syntax, the website first shows how a structure behaves when operations are performed on it. This approach makes abstract topics like stacks, queues, arrays, trees, and tries easier to observe, discuss, and remember.

The project combines animated visualizations, simple real-world analogies, operation controls, complexity summaries, and multi-language code snippets. It is deployed publicly through GitHub Pages so students, teachers, and evaluators can access it directly from a browser.

2. Project Objectives

- To make foundational DSA concepts easier for school students to understand visually.
- To provide interactive controls for operations such as push, pop, enqueue, dequeue, insert, search, and traversal.
- To connect visual behavior with code examples in JavaScript, C, C++, and Java.
- To create a clean and responsive educational website suitable for classroom demonstration and self-study.
- To host the completed project online using GitHub Pages for easy access and presentation.

3. Student And Project Details

Student Name	Aikansh
Registration Number	12303820
Class Section	9P196
Project ID	WPEP-26-284
Live Website	https://aikansh-official.github.io/DSA-Visual-Lab/
GitHub Repository	Aikansh-Official/DSA-Visual-Lab

4. Technology Stack

- **React:** Used to build reusable UI components and manage interactive learning views.
- **TypeScript:** Provides typed development and safer component logic.
- **Vite:** Used as the frontend build tool and development server.
- **Tailwind CSS:** Provides the visual styling system, spacing, layout, and responsive design utilities.
- **Lucide React:** Supplies consistent icons for controls, actions, and concept cards.
- **Motion:** Adds smooth animation for transitions and visual state changes.
- **GitHub Pages:** Hosts the live website as a static production build.

5. System Overview

The application is structured as a single-page React website. The top navigation lets users switch between DSA topics. Each topic page combines a hero explanation, a visualizer, core idea notes, operation buttons, complexity cards, and a code block. The code block includes language tabs so the same concept can be compared across multiple programming languages.

Core modules currently included:

- Stack visualization using a plate-stack analogy for LIFO behavior.
- Queue visualization using a ticket-line analogy for FIFO behavior.
- Array visualization showing indexed contiguous memory cells.
- Tree section covering Binary Tree, Binary Search Tree, AVL Tree, Red-Black Tree, B-Tree, and Trie concepts.
- Premium unlock prompt and multi-language code snippet support.

6. Website Screenshots

The following screenshots were captured from the live GitHub Pages deployment. They show the working user interface and the main educational sections of the project.

The screenshot shows the 'Stack Data Structure' page. At the top, there's a navigation bar with 'Stacks' selected, and a 'Start Learning' button. The page title is 'Stack Data Structure' under the 'Linear Data Structure' category. The main text explains LIFO using a stack of plates. A diagram shows a stack of five plates (Plate 1 to Plate 5) with 'push()' (add to top) and 'pop()' (remove top) operations. Below the diagram is a 'Plate Stack Visualizer' section with a placeholder for an interactive visualizer. To the right, 'The Core Idea' section explains that you can only add or remove the top plate.

Linear Data Structure

Stack Data Structure

Understand Last In, First Out (LIFO) using a stack of plates. A precise, linear data structure essential for parsing, backtracking, and memory management.

push() add to top

pop() remove top

TOP

LIFO MENTAL MODEL

Plate Stack Visualizer

The Core Idea

Imagine a stack of heavy plates in a cafeteria. You can only add a new plate to the top, and you can only remove the top plate. To reach the bottom plate, you must remove all the plates above it first.

Figure 1: Stack page showing the LIFO plate-stack explanation and interactive visualizer.

The screenshot shows the 'Queue Data Structure' page. At the top, there's a navigation bar with 'Queues' selected, and a 'Start Learning' button. The page title is 'Queue Data Structure' under the 'Linear Data Structure' category. The main text explains FIFO using a line of people at a ticket counter. A diagram shows a queue of five people (1 to 5) with 'FRONT' and 'REAR' pointers. Below the diagram is a 'Queue Visualizer' section with a placeholder for an interactive visualizer. To the right, 'Core Idea' section explains that a queue is a linear collection of elements that can be modified by adding or removing elements from the ends.

Linear Data Structure

Queue Data Structure

Understand First In, First Out (FIFO) using a line of people at a ticket counter. A Queue operates exactly like a real-world line: the first person to arrive is the first one served.

FIFO queue first person served first

FRONT

REAR

ENQUEUE AT REAR, DEQUEUE AT FRONT

Queue Visualizer

Status: Initialization

Core Idea

A queue is a linear collection of elements that are maintained in a sequence and can be modified by the addition of entities at one end and the removal of entities from the other end.

Figure 2: Queue page showing the FIFO ticket-line explanation and queue visualizer.

6. Website Screenshots

>_ DSA Visual Lab

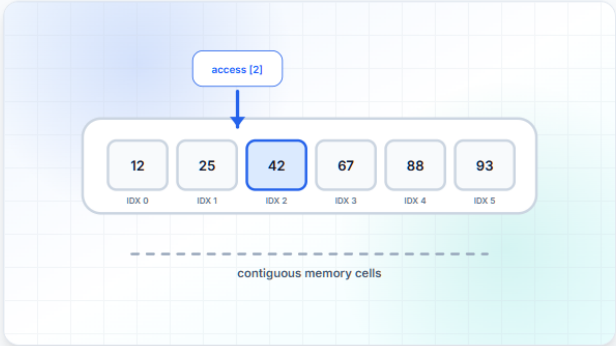
Stacks Queues Arrays Trees

Start Learning

STATIC/DYNAMIC STRUCTURE

Array Data Structure

The most fundamental data structure. A collection of elements identified by index or key, stored in contiguous memory locations for fast access.



contiguous memory cells

Array Memory Visualizer
CONTIGUOUS BLOCKS [0...N]

Status: Initialization

Key Characteristics

- Contiguous Memory**
Elements are stored next to each other.
- Zero-Indexed**
Access starts at index [0].

Figure 3: Array page showing indexed memory cells and array operation controls.

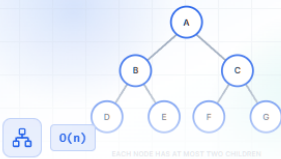
>_ DSA Visual Lab

Stacks Queues Arrays Trees

Start Learning

Tree Architectures

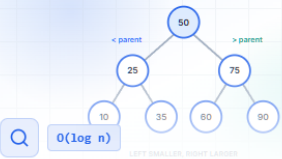
Explore the hierarchical world of trees, from simple binary structures to self-balancing systems optimized for performance.



Binary Tree

A hierarchical structure where each node has at most two children, referred to as the left child and the right child.

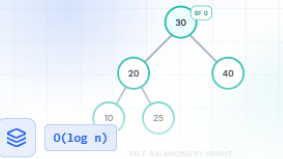
SEE VISUALIZATION



Binary Search Tree

A binary tree where the left child is smaller and the right child is larger than the parent node, optimized for searching.

SEE VISUALIZATION



AVL Tree

A self-balancing binary search tree where the heights of the two child subtrees of any node differ by at most one.

SEE VISUALIZATION

Figure 4: Tree architectures page with relevant visual diagrams for different tree types.

6. Website Screenshots

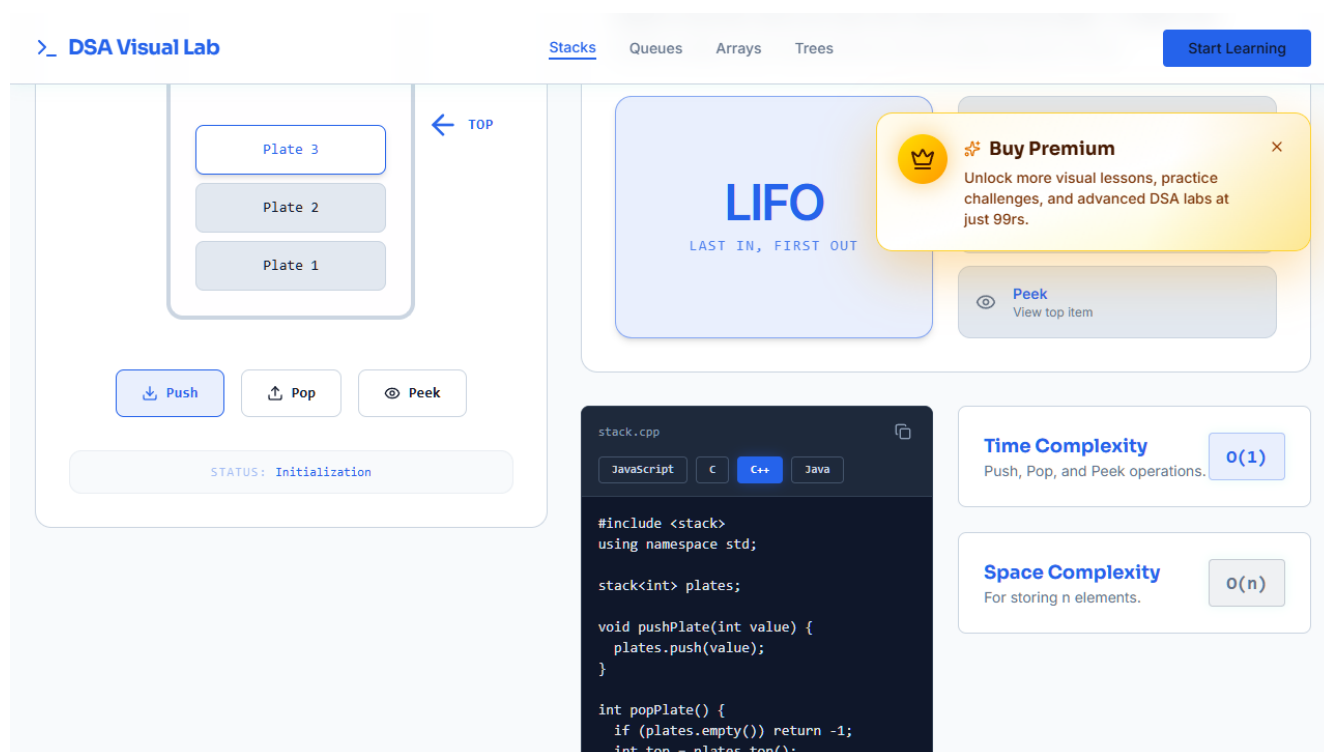


Figure 5: Premium unlock message and multi-language code tabs for JavaScript, C, C++, and Java.

7. Key Features

- Topic-based navigation for fast switching between DSA concepts.
- Animated visual states that show how operations affect each structure.
- Real-world analogies that make abstract concepts easier for beginners.
- Complexity cards for quick revision of time and space complexity.
- Multi-language code examples to support wider programming learning.
- Public deployment through GitHub Pages.

8. Educational Value

The project is especially useful for school students because it introduces the behavior of data structures before showing full code. This visual-first approach supports logical thinking and reduces the fear that beginners often feel when DSA is presented only through syntax.

Teachers can use the website during classroom explanation, while students can use it for revision and self-study. The combination of interaction, animation, code, and complexity summaries makes the website a bridge between textbook learning and practical programming.

9. Future Scope

- Add linked list, graph traversal, heap, hashing, sorting, and dynamic programming visualizations.
- Add quizzes after each visualization to test conceptual understanding.
- Introduce progress tracking for students and a classroom dashboard for teachers.
- Add an AI hint assistant to explain mistakes in simple language.
- Improve metadata, search visibility, and certificate-style project documentation.

10. Conclusion

DSA Visual Lab successfully demonstrates how a modern web application can make core computer science topics more understandable for young learners. By combining visual models, interactive controls, code references, and live deployment, the project shows both educational purpose and practical frontend implementation.

11. References

- Live website: <https://aikansh-official.github.io/DSA-Visual-Lab/>
- GitHub repository: Aikansh-Official/DSA-Visual-Lab
- React, Vite, Tailwind CSS, and GitHub Pages official documentation.